

ISA 与机器级程序

从程序意图到下一架构状态的可执行契约

408 · 计算机组成原理 Topic CO-02

母问题：软件意图怎样成为处理器必须兑现的语义？

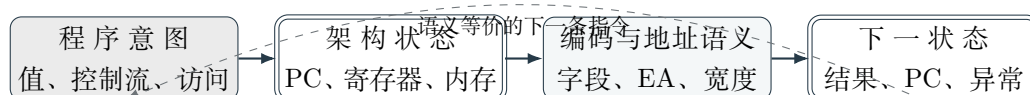
高级语言只说值、控制流和存储访问想要什么；处理器必须把它们编码成有限长度的指令，并规定执行后软件能观察到的下一状态。本册的生成链是

Program Intent → Architectural State → Encoding / Address Semantics → Next Architectural State

箭头首先表示语义约束的生成方向：意图决定所需状态变化，状态变化必须有编码和地址解释，最后形成新的架构状态。它不是某张具体数据通路图的时序；CO-03 才负责把已经确定的语义交给通路与控制实现。

本册定位与 Owner 边界

- **Owns:** ISA contract、架构状态、指令格式与编码预算、寻址和有效地址、访存宽度、字节序/对齐、控制转移、C 到机器级语义、ABI 调用边界。
- **Uses:** CO-01 的有限位宽表示与溢出语义；CO-03 的数据通路和控制；CO-05/06/07 的存储与地址翻译接口。
- **Stops:** 流水线重叠与 hazard 交给 CO-04；Cache/TLB 的实现交给 CO-06/07；OS 的 fault、调度和进程上下文交给 OS 与跨科 Bridge。



目录

1 先固定对象：架构状态不是硬件内部状态	3
1.1 软件契约层	3
1.2 实现层	3
1.3 可访问性是 ISA 相对的	3
2 指令编码是有限预算分配	3
2.1 四个最小问题	3
2.2 扩展操作码的生成	3
2.3 定长与变长、RISC 与 CISC	4

3 从程序意图到可执行映像：四个转换阶段	4
4 寻址：先形成地址，再访问对象	4
4.1 统一生成式	4
4.2 地址、值、地址的地址	5
5 访存宽度、字节序与对齐	5
5.1 字节序	5
5.2 访问宽度与扩展	5
5.3 对齐	5
6 从 C 表达式到机器级状态	6
6.1 数组与表达式	6
6.2 控制流	6
6.3 函数调用：ISA 与 ABI 的分层	6
7 贯穿母例：$x = a[i] + 1$	6
8 不变量、边界与代价	7
8.1 必须保持的不变量	7
8.2 五列概念边界	7
8.3 设计权衡	7
9 与相邻 Owner 的接口	7
10 做题控制协议	8
11 全科坐标与跨册交接	8
12 章节与考纲映射	9
13 一页压缩与自测	9
14 校验依据	9

1 先固定对象：架构状态不是硬件内部状态

1.1 软件契约层

设架构状态为

$$A = (PC, R, M, F, C, \dots),$$

其中 PC 是程序计数器, R 是架构寄存器, M 是软件可观察的内存效果, F 是架构标志, C 表示 ISA 规定的控制/特权状态。指令 I 定义状态转移

$$A_{t+1} = \mathcal{I}(A_t).$$

ISA 的稳定问题是：执行后哪些状态必须改变，哪些状态必须保持不变，以及异常发生时允许观察到什么。

1.2 实现层

IR、MAR、MDR、临时寄存器、流水寄存器和 Cache 元数据可以是微架构状态。它们服务于实现，却不自动成为 ISA 规定的对象。相同 ISA 可以使用不同的寄存器数量、流水级数、内部总线 and 控制器，只要架构状态轨迹与契约等价。

不可逆推的边界

看到教材图上的固定 PC 加法器、MAR/MDR 或某个九拍节奏，不能反推出所有 ISA 都必须有这些部件。ISA 定义“必须发生什么”，CO-03 才讨论“一种实现怎样让它发生”。

1.3 可访问性是 ISA 相对的

旧笔记的“用户可见/部分可见/系统可见/完全不可见”适合作为提问方向，不是跨架构固定清单。稳定判据是：当前 ISA 是否提供读取、写入、配置或间接影响该状态的合法操作？例如 PC 可能被跳转间接改变，页表基址可能有特权接口，PCB 通常是 OS 软件对象而非必然 CPU 寄存器；Cache 对普通指令透明，却可能有特权管理操作。

2 指令编码是有限预算分配

2.1 四个最小问题

每条指令必须回答：**做什么** (opcode)、**对谁做** (操作数/地址)、**结果放哪** (目的状态)、**下一步去哪** (顺序 PC 或控制转移目标)。地址字段数量形成零地址、一地址、二地址、三地址等表示，但地址数量不是性能或表达能力的单一排名。

2.2 扩展操作码的生成

若指令字长为 L ，本层保留 w 位地址字段，操作码拥有 $L - w$ 位。已经使用的编码不能再分配，剩余前缀才可以作为下一层的 escape：

$$N_{k+1} = (N_k - U_k)2^{w_k},$$

其中 N_k 是本层可用编码数， U_k 是本层已经占用的指令数， w_k 是被释放给下一层的字段位数。

16 位字、6 位地址字段

二地址层保留 4 位操作码，共 16 个编码。若已用 15 个，剩下 1 个作为扩展前缀；一地址层得到 $1 \times 2^6 = 64$ 个候选编码，若已用 34 个，零地址层只从剩余 $30 \times 2^6 = 1920$ 个编码中继续展开。每一层都必须同时检查“剩余数量”和“前缀无歧义”，不能把 escape 当普通指令再算一次。

2.3 定长与变长、RISC 与 CISC

定长格式让字段位置和取指边界规整，便于译码和并行取指；变长格式可以提高代码密度，却增加边界识别和译码成本。RISC/CISC 是一组设计权衡：指令规整度、内存操作约束、代码密度、译码复杂度、编译器与硬件分工都可能参与，不能推出“RISC 必然硬布线”或“CISC 必然微程序”。

3 从程序意图到可执行映像：四个转换阶段

“高级语言能运行”不是编译器一步把文字变成 CPU 动作。稳定的生命周期是：

source $\xrightarrow{\text{compiler}}$ assembly/relocatable object $\xrightarrow{\text{assembler}}$ machine object $\xrightarrow{\text{linker}}$ executable image $\xrightarrow{\text{loader}}$ process address

编译器把表达式、控制流和函数调用映射为 ISA 可表达的指令与数据布局；汇编器把助记符和伪指令解析为机器码、符号表和重定位记录；链接器解析跨目标文件的符号引用，合并代码/数据段并完成或保留重定位；装入器把可执行映像放入进程地址空间，设置入口 PC、栈和必要的动态绑定。每一步都可能改变表示，但必须保持最终可观察语义。

这条链也解释了三个边界：编译器选择不等于 ISA 必需序列；链接地址不等于运行时物理地址；装入完成不等于第一条指令已经提交。题目给出目标文件、符号或重定位项时，先判断它处在哪一个阶段，再决定谁能修改它。

阶段	主要输入/输出	不能直接推出的事实
编译	源语言语义 → 汇编/目标代码	唯一指令序列或固定寄存器分配
汇编	助记符/伪指令 → 机器码、符号、重定位	所有地址已经最终确定
链接	多个目标文件 → 可执行映像	已经位于某个进程的物理内存
装入	映像 → 进程地址空间与入口状态	已经完成执行或不会缺页

4 寻址：先形成地址，再访问对象

4.1 统一生成式

有效地址（effective address, EA）是地址形成阶段的输出：

$$EA = f(\text{instruction fields}, R, PC, M).$$

它回答“对象在哪里”，不等于已经读出了对象。若系统启用虚拟地址，EA 还要经过 CO-07 的翻译接口；若访问 Cache，则继续进入 CO-06。

方式	操作数/地址来源	数据访问	语义与题面信号	常见误判
立即	字段就是值	0	常量、符号扩展、位数限制	把立即数当地址
寄存器	R_i 中是值	0	算术和寄存器间数据流	把寄存器值再算一次 EA
直接	字段给 EA	1	形式地址直接定位对象	忽略地址空间/编址单位
寄存器间接	$EA = R_i$	1	指针解引用	把指针本身当对象
基址 + 偏移	$EA = R_b + \text{sext}(d)$	1	栈帧、结构体、重定位	忘记偏移扩展/字节编址
变址/比例	$EA = B + R_i K + d$	1	数组元素和记录数组	忘记 $K = \text{sizeof}(T)$
PC 相对	$target = PC_{base} + d$	取指路径	分支、位置无关代码	不确认 PC 基准和位移单位
间接	先取地址再解引用	≥ 2	多级指针或指针表	把“取地址”与“取数据”混写

4.2 地址、值、地址的地址

做题时把三种量分行写：

$$\text{value} = M[EA], \quad \text{pointer} = M[EA], \quad \text{next address} = M[M[EA]].$$

访存次数必须从这些层级推出来，而不是从寻址名称背口诀。地址形成是一次算术关系；访存是对该地址对应存储对象的读取或写入。

5 访存宽度、字节序与对齐

5.1 字节序

Endian 只规定多字节对象的字节怎样映射到递增内存地址，不改变该对象在寄存器中代表的数学值。设一个 32 位值由四个字节组成，题目必须先画地址表，再判断低地址存放最高有效字节还是最低有效字节；不能按十六进制字符串的视觉顺序猜。

5.2 访问宽度与扩展

Load 的宽度决定读几个字节；signed/unsigned load 决定读入寄存器时如何扩展。Store 只写 ISA 规定宽度的低位或对应字节，不因为寄存器更宽就自动写满整字。地址宽度、指针宽度、PC 宽度、指令长度和物理地址线数属于不同预算，题设没有等式时不能强行相等。

5.3 对齐

若对象大小为 k 字节，常见对齐条件是地址满足 $a \equiv 0 \pmod k$ ，但 ISA 可以允许未对齐访问、将其拆为多次事务，或触发异常。答题必须写清“题设 ISA 的允许行为”，不能把某一实现的惩罚当成普遍语义。

6 从 C 表达式到机器级状态

6.1 数组与表达式

对元素类型大小 $s = \text{sizeof}(T)$ ，一维数组地址为

$$\text{addr}(a[i]) = \text{base}(a) + i \times s.$$

行优先二维数组为

$$\text{addr}(a[i][j]) = \text{base} + (iN + j) \times s.$$

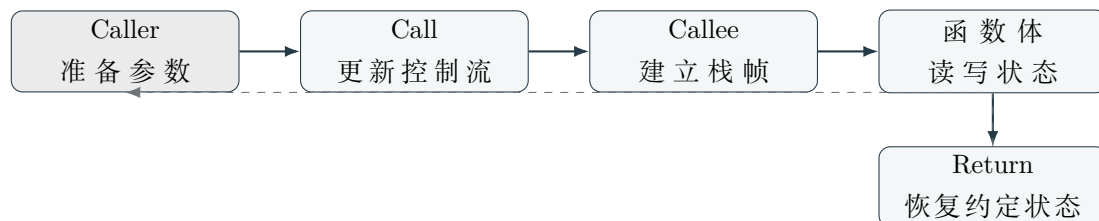
这条地址生成式连接三个 Owner：CO-02 负责语义和 EA，CO-06 负责局部性/副本命中，CO-07 负责 VA 到 PA 的翻译；不能在 CO-02 里重新讲 Cache 或页表机制。

6.2 控制流

条件结构通常变为比较、条件选择下一 PC 和无条件跳转形成回边。分支目标可以是 PC 相对、寄存器间接或 ISA 规定的其他形式；比较是否产生显式标志、是否与分支合并，由 ISA 决定。

6.3 函数调用：ISA 与 ABI 的分层

函数调用至少有四个状态交接：参数/返回值位置、返回地址、保存责任、栈帧布局。ISA 提供跳转、寄存器和访存等原语；ABI 约定哪些寄存器 caller-saved/callee-saved、参数何时溢出到栈、返回值放在哪里。ABI 不是硬件自动保存规则，具体名称也不能跨架构照抄。



7 贯穿母例： $x = a[i] + 1$

设 a 是元素大小为 s 的数组， i 在寄存器中，目标 x 可能是寄存器变量或内存对象。先写架构状态差，而不是先背某套汇编：

$$EA = R[\text{base}(a)] + R[i] \times s,$$

$$v = M[EA],$$

$$R[x]' = v + 1 \quad \text{或} \quad M[\text{addr}(x)]' = v + 1.$$

随后才选择一条具体 ISA 的指令序列：计算偏移、形成 EA、load、加立即数、必要时 store。编译器的寄存器分配和优化可能改变序列，但必须保持可观察语义一致。

母例中的典型错误路径

“ $a[i]$ 已经是一个值，所以直接把 $\text{base} + i$ 当地址”漏乘了元素大小；“load 结束，所以 x 已经提交”又把中间值与架构写回混淆。正确顺序是：先确认对象和表示，再形成 EA，再区分产生值与写入目标状态。

8 不变量、边界与代价

8.1 必须保持的不变量

- 同一编码在当前 ISA 下语义确定，不能因某张数据通路图不同而改变；
- 指令允许改变的 Write Set 之外，其他架构状态保持不变；
- 地址形成、访存宽度、扩展和对齐必须与 ISA 规定一致；
- 调用者与被调用者对 ABI 保存责任一致；
- 异常或 fault 发生时，架构可见效果符合该 ISA 的精确性规则。

8.2 五列概念边界

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
ISA	≠ 微架构	ISA 规定软件可观察语义；微架构选择路径、时序、Cache 与控制器	用教材通路否定另一合法实现
EA	≠ value	EA 定位对象；value 是读取对象后的内容	少算/多算一次访存
ISA	≠ ABI	ISA 是机器契约；ABI 是编译模块间的调用约定	把 caller/callee 保存误当硬件自动动作
字节序	≠ 位权解释	字节序描述内存地址顺序；位权描述位串数值解释	反转寄存器值或地址表
对齐	≠ 地址宽度	对齐是合法起始地址约束；宽度是可表示地址/数据范围	从 32 位指令推出 32 位地址或固定异常
PC 相对	≠ 绝对地址	相对目标由 PC 基准和位移形成；绝对地址直接给出位置	分支范围和重定位判断错误

8.3 设计权衡

固定格式、更多寄存器、较大立即数、更多 opcode 会竞争同一编码预算；更复杂的寻址可以减少显式指令，却增加译码和地址形成复杂度；Load/Store 限制让数据通路规整，却可能需要更多指令；寄存器传参减少访存，但寄存器不足时仍需栈帧。任何“更快/更省”的结论都要说明比较对象和成本项。

9 与相邻 Owner 的接口

- **CO-01:** 提供位串解释、有限宽度结果和异常/溢出语义；CO-02 只调用它，不重讲补码电路。
- **CO-03:** 接收已确定的 Read Set、Derived Values、Write Set 和 Next PC，生成通路、调度与控制字。
- **CO-04:** 接收单指令依赖和语义，负责多指令时间重叠、forwarding、stall、flush 与 ordered commit。
- **CO-06/07:** CO-02 只输出访问宽度、EA/VA 和重试语义；Cache 命中状态、TLB/page walk 和页表硬件不在本册拥有。

- **OS / X-B01**：异常入口、特权切换和 handler 的软件动作是跨层接口；不能把“用户不可见”简化成“所有特权软件都不可见”。

10 做题控制协议

看到指令格式、寻址、机器级代码、栈帧或字节序题，按以下顺序执行：

1. 写题设 ISA、编址单位、指令长度和操作数宽度；缺失时列出不能强推的量；
2. 写指令前后的架构状态差：Read Set、Write Set、Next PC、异常可能性；
3. 解码字段预算，明确 opcode、register、immediate 和 function 各占哪些位；
4. 对每个内存操作先写 *EA* 生成式，再区分 value、pointer、address-of-address 和访存次数；
5. 若是数组/栈帧，先乘元素大小或确认偏移单位；若是多字节对象，画地址—字节表；
6. 函数题分开 ISA 原语、ABI 约定和编译器选择；不要把某架构寄存器名称当成普适规则；
7. 最后把语义交给 CO-03/CO-04，独立检查状态差、地址单位、写回位置和异常可见效果。

压缩成第一动作

题面出现“指令格式/寻址/机器代码/调用/大小端”时，先问：**这条指令要改变哪个架构状态？操作数现在是值、地址还是地址的地址？**写出状态差和 *EA* 后，剩余计算才有唯一落点。

11 全科坐标与跨册交接

Map Coordinate: ISA State → Data Location

CO-02 先写指令前后的架构状态差，再由编码、寻址和访问宽度说明操作数如何被定位。输出是 CO-B01 所需的语义包与 CO-03 可实现的读/写集合，不把某套微架构的内部节拍升级为 ISA 事实。

本册局部成本来自编码预算、指令长度、*EA* 生成、对齐/字节序处理与 ABI 约定；它们可能改变取指、访存和调用开销，但程序总时间仍需结合 IC、CPI、时钟周期及 Cache/Memory 行为。

12 章节与考纲映射

传统知识块	本册的机制位置	Stop Boundary
指令系统与指令格式	编码预算、字段竞争、扩展操作码	具体控制器实现归 CO-03
寻址方式与有效地址	EA 生成、地址/值分层、访存次数	翻译硬件归 CO-07
机器级程序与 C 映射	数组地址、控制流、函数调用、栈帧	优化器内部选择只作实现例子
RISC/CISC	代码密度、规整译码、内存访问和软硬件分工的权衡	不推出控制器一一对应
字节序、对齐、访问宽度	地址表、扩展、合法访问边界	平台特例必须由题设声明

13 一页压缩与自测

一句话

ISA 先规定“程序执行后软件必须看到什么”；机器级程序把意图编码成状态变化，寻址把字段解释为对象位置，ABI 把调用双方接起来，而 CO-03 再决定这些语义怎样由硬件路径实现。

忘掉公式后，尝试回答：

- 1. 为什么指令长度、寄存器数量和立即数字段会竞争同一预算？
- 2. 为什么 EA 不是 value？间接寻址为什么多一次访存？
- 3. 为什么 32 位指令不能推出 32 位 PC 或物理地址？
- 4. 为什么函数调用中的 caller-saved/callee-saved 属于 ABI 而非普适 ISA 自动动作？
- 5. 给定 $x = a[i] + 1$ ，能否从状态差独立重建 EA、load、加法和最终写回？

14 校验依据

本册吸收了 CO-程序机器级表示与指令系统设计、CO-函数调用栈帧、CO-寄存器按可访问性分类以及归档指令格式/寻址/指令周期 Source 的可迁移部分。MIPS 固定格式、某套 ABI、固定节拍和固定寄存器分类仅作为例子或被拒绝的过度推广；后续规则验证以真实 408 题和陌生 ISA 题为准。